

Propriétés CSS utilisées :

- Les variables CSS
- Les animations / transformations CSS
- Les flux et les débordements de texte
- Les pseudo-classes CSS
- La visibilité en CSS

Pourquoi :

1. Pour les variables CSS c'est simplement un gain de temps et une amélioration de l'organisation. Cela permet de déclarer une fois une valeur et de l'appeler lorsque l'on le souhaite. Cela évite des fautes de frappe, des petites différences entre certaines couleurs et autre.
2. J'ai utilisé les animations et transformations CSS pour rendre le tout plus esthétique et plus agréable à lire. C'était une vraie galère à mettre en place et à comprendre, mais finalement le résultat est très satisfaisant et permet une lecture vraiment agréable et fluide.
3. J'ai utilisé la propriété de débordement de texte (overflow-y : hidden) sur le body et les media queries. En fait, je me trouvais face à un problème, si je définissais « overflow-y : hidden » sur le body, si la largeur de l'écran était supérieure à 1200px tout fonctionnait, cependant avec une valeur plus basse, le scroll était impossible et certains éléments se retrouvaient coupés et/ou on ne pouvait pas accéder à la suite du formulaire. A l'inverse, en ne définissant pas « overflow-y : hidden » sur le body, cela fonctionnait, toutefois je me trouvais, au-dessus de 1200px de large, avec des interfaces scrollables sans qu'on en ait besoin. C'est pour cela que j'ai fini par utiliser les media queries pour dire qu'entre 120px et 1200px de large l'interface est scrollable, et au-dessus elle ne l'est plus.
4. J'ai utilisé les pseudo-classes CSS pour rendre le tout plus joli et agréable. Par exemple avec :hover, les boutons changent lorsque l'on les survole, ou alors avec :focus, j'ai fait en sorte que les textarea/inputs changent lorsque l'on appuie dessus.
5. Pour finir j'ai utilisé la visibilité en CSS pour les inputs, ce qui permet de différencier les placeholder des valeurs que l'on a inscrites (opacity), ou alors visibility sur le second logo qui est caché pour bricoler un header à-peu-près fonctionnel.

Les bénéfices :

- Une meilleure structure de code (variables)
- Une interface agréable à utiliser (animations, (pseudo-classes))
- Une navigation fluide et intuitive (animations, pseudo-classes, visibilité)
- Une flexibilité de la taille de l'écran fonctionnelle (débordement)

J'ai utilisé d'autres propriétés que je n'ai pas cité, comme les styles de texte, les interactions et sélections, les listes, et cibler les éléments avec les combinateurs (uniquement avec :not).